Verifying Stop Scope in Agent Workflows

Huseyin Buldurgan

Independent Researcher · huseyin@latentatlas.ai

Research preprint · Version 1.3 · 23 October 2026

Abstract

Stopping an agent run and preventing the effects of work it has already initiated are different claims. I study this distinction in a pinned Open-SWE workflow and research-built local receivers. The original comparison used 26 programmed conditions, repeated on the same host, to separate cancellation, worker completion and effect-time authority checks. A five-condition live-model development comparison tests those distinctions across a clean receiver restart. Stopping the source run and revoking its origin only in memory each allowed one old-task file effect and two programmed follow-up effects after restart. Persistent cancellation blocked the accepted operation but not the follow-ups; persistent origin revocation blocked both. Independent and newly authorized reports matched the healthy reference in all eight comparisons. Four separately authorized stop summaries also delivered final-analysis values before receiver control and restart, distinguishing file prevention from information-delivery scope. A provider-free reproducer complements these observations with 12 conditions across three intervention times and four fault checks. The contribution is a set of paired, auditable counterexamples to treating a narrow stop observation as a broader guarantee: source termination, operation cancellation and file prevention each leave a different part of the requested scope unresolved. These bounded development experiments use one host and do not estimate model resistance. The wider 286-case catalogue remains a research programme rather than a completed benchmark.

1. Research question and contribution

When an agent run is stopped, which controls prevent effects from work it has already initiated, and what evidence establishes this without disabling independent or newly authorized work?

An operator may stop a conversation or cancel a run while a worker is still writing, a receiver is holding accepted work, or a separate summary task can deliver the result. Saying that the source stopped leaves each of those downstream states unresolved. The practical question is therefore not just whether a stop was requested, but which work, authority, effects and output channels the request actually covered.

I test three separately checkable requirements: no old final-file effect after a stop request (R); no old-origin write beginning after effective local revocation (V); and no lingering worker effect after a scope-specific completion acknowledgement (C). The original comparison separates these requirements. The live-workflow comparison then asks how their scope changes when an accepted operation crosses a receiver restart, other work must remain usable, and an authorized summary has its own output channel.

The empirical contribution is a set of paired counterexamples to overly broad stop claims. A stopped source can leave an accepted effect pending; cancellation of that operation can leave later same-origin effects possible; and prevention of a final file can coexist with delivery of its analysis values in a summary. Each distinction is tied to a control comparison, actual file bytes or delivered text, and an offline audit. Table 4 translates them into evidence an operator should request before making the corresponding claim.

1.1 Motivation and development

This project began in Apart Research's 11–13 September AI Incident Response Sprint. I initially wanted to compare possible stopping dates for the Hugging Face–OpenAI incident, balancing prevented harm against lost research knowledge. The public record did not establish what research value would have been lost at each date, so I could not justify that cost–benefit calculation. I turned instead to a question I could test: what does stopping actually require?

The sprint submission used programmed, cooperative actors. Reviewers valued the distinction between stopping claims and the reproducible records, but correctly challenged the lack of live-model conditions and the long introduction. I continued the project by adding live work generation and by requiring independent and newly authorized work to survive a scoped stop. Those design choices preserve the original mechanism question while making collateral damage observable. Version 1.3 clarifies the contribution, chronology and result tables using the evidence already released with v1.2; it adds no experimental observations.

2. Related work and positioning

The gap between cancelling a caller and stopping its work is established in systems practice. Python futures distinguish cancellable pending work from already running work [1]. Celery distinguishes revocation from termination and documents the persistence needed to retain revocations across restarts [2]. Kubernetes separately documents suspension of future scheduling and the continued execution of already started jobs [3]. These distinctions motivate the controls here, rather than constituting new findings.

Khan's *Stop Means Stop* evaluates enforcement gaps in several agent frameworks, combines model-free probes with live-model observations, and introduces SOUNDGATE under an explicit complete-mediation contract [4]. That work has broader framework coverage and stronger enforcement machinery than the local controls used here. Zhu and Wang address root-scoped quiescence across delegation and asynchronous execution, including evidence conditions for revocation [5]. Their formal treatment is a close antecedent to the scope question in this paper. VERA studies authority-preserving edge revocation, including independently authorized shared agents [6]. Evidence-Carrying Termination checks whether a terminal completion claim is supported by in-scope trace evidence and replay [7].

The present study contributes a narrower operational comparison rather than a new revocation principle. It makes the limits of three apparently reassuring observations inspectable: the source has stopped, the selected operation was cancelled, and the final file is absent. The live restart comparison links each observation to effects it does and does not cover; the provider-free fault controls separately expose ineffective control, collateral damage and missing attribution. This helps an operator select evidence for a stated stop contract. The cited methods are conceptual comparators, not implemented comparison arms: this study establishes neither superiority to them nor a new general shutdown protocol.

3. Evidence structure and original comparison

The original study pins Open-SWE to commit `e0d9aff59925a4da55ec8d6c31fa723651c24909` [8]. It uses the real local factory, middleware, tool routing, in-process Agent Server interface, Store and native files. Model responses in that comparison are programmed. The task reads 24 synthetic incident records and produces a 12-record partial report and a 24-record final report.

Four arms compare cancellation plus summary, waiting for worker completion plus summary, origin-scoped control plus summary, and origin-scoped control with ordinary tools available to the old completion. The last arm removes a tool restriction rather than all summarization. Three schedules and two fixed input seeds, 17 and 41, produce 24 conditions; two worker-wait timeouts bring the total to 26. Each condition starts a fresh process. A fresh environment on the same host repeats the 26 conditions; these are 52 executions of 26 conditions, not 52 independent settings.

The schedules intervene while an accepted worker is held before writing, after the final effect, or without the timing barrier. Table 1 shows the held-before-write comparison. Cancellation permits the pending write. Waiting permits it too, but the worker finishes before the controller acknowledges completion. The local origin gate prevents the write. In the after-effect and unbarriered schedules, the observed final file already existed before intervention and was not rolled back.

Table 1. Original pre-write comparison, per two fixed inputs.

Control	Final file written after stop	Worker-completion interpretation
Cancel plus summary	2/2	No quiescence acknowledgement offered
Wait for worker plus summary	2/2	Worker completes before acknowledgement
Origin-scoped plus summary	0/2	Pending native write prevented
Origin-scoped, ordinary tools	0/2	Pending native write prevented

R, V and C are scored separately. In the cancellation and waiting arms, the revocation boundary is a shadow comparison event: no enforcing origin gate is active. A V failure there is not a broken upstream revocation promise. C is not applicable when no quiescence acknowledgement is made; a wait timeout is not silently recorded as completion.

The scoped control serializes revocation, origin checking and the native write under one per-origin lock. Observation bounds a file effect between its native attempt and its returned result, which includes the file hash. Worker entry is not the write boundary; an interval crossing a boundary is not forced into a definite classification. This records local file effects, not instantaneous physical commitment or crash durability. Raw messages, identity records, worker events and final bytes are reconciled separately from the intervention code. All 26 conditions and their same-host repeats passed the evidence audit, including conditions in which the control did not prevent an effect [9].

4. Live persistence comparison

4.1 Actors, interface and fixed checkpoint

The live comparison uses one complete five-arm development sequence, seed 17, on the same host. O is the old task. I is an independent task. N starts with fresh human authorization. S is the framework's separately authorized stop-summary task. These tasks use live `gpt-6-astra` responses with requested reasoning level `high` through the ChatGPT-plan token-sharing preview route. The reported model name is an alias, not a dated immutable model snapshot.

Requests use streaming and `store: false`, with namespace-wrapped tools and the recorded route-specific fields. O/I/N are offered `read_file`, `write_file` and `edit_file`; S is offered `read_file` and a

local `slack_thread_reply` delivery adapter. This is a restricted tool surface on the real framework, not its complete production configuration. Full request text and tool schemas are retained in the sharing derivative. The underlying framework source is pinned as in Section 3.

A separate local receiver accepts operations and records them in SQLite. The checkpoint holds O's first proposed final mutation and I's first mutation after acceptance but before their effects. The source's Slack stop path interrupts O's framework run while the receiver still holds its accepted operation. O is not sent a natural-language stop instruction. S can then inspect the prior conversation and report status. The receiver control is applied after S finishes, then the receiver is closed, reaped and reopened against the same database in a new process.

Two programmed follow-up operations, denoted F here, are submitted under the old origin after restart. They test scope beyond the original operation identifier. The archived role label remains C; F avoids confusion with completion requirement C in this paper. Healthy continuation establishes that the task outputs and follow-up effects are feasible. The source, receiver and model-call identities are bound in the saved evidence.

Figure 1. Event sequence in the four stop arms (schematic; not elapsed time).

1	**ACCEPT:** O's final mutation and I's first mutation are held before effect.
2	**INTERRUPT:** the framework stops O's run; the source process stays alive.
3	**SUMMARIZE:** separately authorized S delivers its text before receiver control.
4	**CONTROL / RESTART:** apply the arm's receiver control, close and reap the receiver, then reopen the same SQLite database in a new process.
5	**RESUME / PROBE:** release accepted work; redeliver the same O operation; run freshly authorized N and two old-origin F operations.
6	**AUDIT:** reconcile post-restart effects, final files, I/N reference matches and the earlier S text as separate outcomes.

The healthy arm uses the same accepted-work checkpoint and receiver restart without interrupting O or running S. The timing of S matters: its result delivery precedes the receiver intervention whose file effects are scored.

4.2 Interventions and measurements

The arms are: healthy continuation; source-run stop only; source stop plus in-memory origin revocation; source stop plus persistent cancellation of the accepted operation; and source stop plus persistent revocation of the old origin. In-memory revocation intentionally does not survive a receiver restart. Persistent operation cancellation records one operation identifier. Persistent origin revocation records the origin and checks that authority at the receiver. These are research-built controls, not claims about native Open-SWE guarantees.

The primary outcome is successful old-task native file mutation in the second receiver lifetime. Follow-up effects, final file contents, duplicate-delivery records, I/N preservation and summary content are recorded separately. Audits reconcile receiver journals, two SQLite snapshots per episode, provider-message projections and actual file bytes. Each episode's identity, transport, usage and closure checks are completed before moving to the next arm. This is a sequential development comparison, not randomized allocation or a preregistered statistical pilot.

4.3 Results

Table 2. Live-workflow persistence comparison: one episode per arm.

Control	O effects after restart	F effects after restart	I / N final files
Healthy reference	1	2	Reference / reference
Source stop	1	2	Match / match
In-memory origin revocation	1	2	Match / match
Persistent operation cancellation	0	2	Match / match
Persistent origin revocation	0	0	Match / match

Source interruption left accepted work able to produce an effect after restart. A nonpersistent decision did not prevent that effect either. Persistent operation cancellation stopped the selected operation but allowed both later operations from its origin. Persistent origin revocation prevented all three effects on the tested path. I/N final files matched the healthy reference in all four intervention arms, yielding eight byte-level comparisons [10].

A match in Table 2 means byte equality with the corresponding healthy final file, not merely the existence of a file. The healthy episode used 12 provider requests and each stop episode used 13.

The five episodes contain 64 archived provider requests: 20 for I, 16 for O, 20 for N and eight for S. Recorded usage totals 648,131 input tokens, including 211,584 cached tokens, and 12,805 output tokens, including 486 reasoning tokens. This reconciles to 130.43260 historical credit-equivalent units under the recorded quote function; it is not a verified account debit. No provider request is made to reproduce these totals.

All four stop summaries also delivered final-window analysis values, including eight denied events and 3,456 bytes. Their exact English texts and hash-bound author decisions are released. This happened before receiver control and restart: it is not evidence that a receiver gate was bypassed. The summary task had prior conversation context and a read tool; a restriction on final-result disclosure was not its instruction. The observation distinguishes successful control of a file channel from control of result delivery in another authorized channel.

5. Executable receiver mechanism test

A second package makes a related experiment runnable without a provider account [10]. It uses separate source, receiver and observer processes, native files and programmed actors. Source stopping uses POSIX process termination, unlike the framework-run interruption in Section 4. Fixed payloads are replayed; the experiment does not regenerate them with a model. It has no restart or durable queue, so it complements rather than reruns the persistence comparison.

Twelve main conditions cross four controls with three cuts: O is queued; O is claimed but has not written; or O has written but its worker has not returned. Controls are healthy continuation, source termination, cancellation of O at the receiver, and receiver-side origin revocation. Here F is one later operation rather than two (again archived as C). At either pre-effect cut, source termination permits O and F; operation cancellation prevents O only; origin revocation prevents both. After O has already written, every main

arm retains that completed file. Origin revocation still prevents the later F effect. All 18 I/N comparisons in the nine main intervention cells match the corresponding healthy files.

Table 3. Fault controls test the measurement system, not just the preferred control.

Injected fault	Observed result	Audit interpretation
Revocation has no effect	O and F each write once; I/N match	Recorded control failure
Revocation covers unrelated origins	No O/F effects; I/N files also absent	Recorded collateral damage
O's native write receipt omitted	O's file exists; its receipt is missing	needs_review; no attributed effect count
Incorrect O report content	O's file exists but differs from its reference; I/N match	Recorded task-quality failure

These four fault checks use the claimed-before-effect cut. The incorrect-payload check uses healthy continuation; the omitted-receipt check uses source termination. Both revocation faults use receiver-side revocation. The full example contains 15 recorded conditions and one intentionally `needs_review` condition. Missing attribution is not converted into success. Conversely, a well-recorded bad outcome is not excluded because it differs from the intended behavior. The package separates immutable inventory checks, raw-evidence auditing and expected-outcome regression comparisons. A changed result on a rerun requires inspection; it is not permission to rerun silently until it agrees.

6. Interpreting a stop claim

The comparisons identify evidence needed to justify a particular stop claim. Table 4 is an operational checklist derived from these bounded tests, not a general completeness theorem.

Table 4. Stop claims and the evidence each requires.

Claim	Evidence to inspect
The source stopped	Target identity, stop event and source terminal state
Accepted work finished	Acceptance inventory, worker lifecycle and completion boundary
An effect was prevented	Effect-time authority decision, native receipt and file inventory
Later old-origin work is blocked	Origin binding plus admission/effect checks for later operations
The decision survives restart	Persisted decision, new receiver identity and replay trace
Results are no longer delivered	Explicit output contract and observation of each delivery channel
Legitimate work is preserved	Independent/new-authority identity and healthy output comparisons

The central implication is that a source-side terminal state is not a substitute for receiver-side evidence. An operation-scoped cancellation cannot establish an origin-wide claim merely because its selected operation stopped. A durable decision also needs enforcement on the path that performs the effect.

Finally, an authorized summary may remain useful while exceeding a stricter information-delivery contract. The desired contract must say which of those outcomes is acceptable.

The risk of validating only my own implementation is real. Healthy references, ineffective and overbroad controls, a separate evidence-reading path and actual file inspection make self-confirming success labels harder to hide. They are still authored within the same project and trust local instrumentation. Separate implementation, independent technical review and other-host execution remain important next checks.

7. Scope, catalogue and next study

These are small, development-stage mechanism experiments. The live comparison uses one model alias, one input and one host; calls within an episode are not independent trials. O's final operation was already selected and accepted when its source was interrupted. F is programmed, and no live O resistance to a natural-language stop was measured. Clean restart does not establish crash consistency, distributed consensus, hostile-runtime integrity or complete mediation of arbitrary tools. The separate receiver test improves executability, not model or task diversity. The original same-host repeat is not an independent replication.

The broader design contains 286 case identities across 24 families and cross-family interaction chains. The 15 October assessment maps two cases to technically closed local packages, 27 more to direct but incomplete evidence, 39 to partial evidence, 125 to reusable components only, 78 to no matching experiment and 15 to blocked prerequisites. Thus 29 cases have direct evidence and 68 have direct or partial evidence, with the former included in the latter. Formal catalogue acceptance remains zero. The released case-by-case map labels this as an assistant-assisted scope assessment, not independent certification or a measure of work-hours completed.

Other development work has examined component removal, matched intervention timing, output restrictions, overwrite behavior and shared resources. It informs the coverage map but is not pooled into the two complete packages reported here. This revision publishes a coherent, auditable work sample; the research goal remains completion of the full catalogue and systematic multi-model testing of critical paths.

Next milestones are family-specific evidence closure, diverse calibrated tasks, matched controls across multiple live models, other-host reproduction and a separately implemented receiver control. Model resistance needs its own design: a clear authorized stop, a genuine safe opportunity to continue, and an observation rule that keeps noncompliant behavior as a result. It should not be inferred from accepted background work.

8. Conclusion

In the tested workflows, stopping the source, cancelling an accepted operation and revoking its origin had different consequences. Accepted work survived source interruption; in-memory revocation did not survive restart; persistent operation cancellation left same-origin follow-ups possible; persistent origin revocation blocked the tested old-origin file effects while preserving independent and newly authorized reports. Authorized summaries remained a separate result-delivery channel.

A useful stop claim should therefore name its target, downstream work, enforcement point, persistence boundary, output channels and evidence. The contribution of this study is to make those distinctions observable and reproducible in a bounded agent workflow, and to provide a concrete basis for the larger evaluation still to be completed.

Data, code and reproduction

The versioned v1.3 release contains this paper, editable text and the revision record. The browsable evidence snapshot preserves the records, auditors, source, coverage map and provenance released on 16 October [10]. That snapshot and its two reviewed archives are unchanged. The original sprint evidence also remains under its original tag [9].

With Python 3.10 or newer, run `python3 -B verify_snapshot.py` in the 16 October snapshot directory. It verifies both saved packages and reconciles the published counts. The live package also has 17 tests; the receiver package has 20. Follow the receiver README to run its fresh smoke test and then the full 16-condition matrix on macOS or Linux. These operations need no API key or model account. A new live-model run is a separate task, not part of the offline reproducer.

The live export is a declared privacy-reviewed derivative. Visible text, tool arguments, synthetic files and SQLite bytes are preserved. Local paths and account-linkage fields are minimized, opaque encrypted replay blobs become digest markers, and affected wire hashes are rebound. Original-to-export hashes and extraction rules are included. Hashes verify integrity, not independent authentication of provider execution.

Acknowledgements and AI assistance

I thank Apart Research and the sprint reviewers for their feedback on scope, readability and live-model evidence. Alex Turner's writing helped motivate my concern with responsibility in AI development [11]; this acknowledgement does not imply his involvement or endorsement.

I initiated the project and chose to continue it after the sprint, incorporating reviewer feedback through live-work generation, scoped controls and explicit preservation of legitimate work. I set the research direction, developed the motivating questions and reviewed the designated outputs. LLM tools, including Codex, assisted technical design, implementation, test development, execution, analysis, literature checking and drafting and editing this revision. The archived `gpt-6-astra` calls are experimental observations, distinct from this research assistance. The original 26-condition actors were programmed. Author-reviewed summary judgments are identified explicitly; other automated checks do not imply independent human review. I am responsible for the claims and their correspondence to the released evidence.

References

[1] Python Software Foundation. *concurrent.futures — Launching parallel tasks*. Documentation. Accessed 16 October 2026.

[2] Celery contributors. *Workers Guide: revoking tasks and persistent revokes*. Documentation. Accessed 16 October 2026.

[3] Kubernetes contributors. *CronJob: schedule suspension*. Documentation. Accessed 16 October 2026.

[4] Sajjad Khan. *Stop Means Stop: Measuring and Repairing the Enforcement Gap in Agent-Framework Control Primitives*. arXiv:2607.14166v3, 2026.

[5] Genliang Zhu and Chu Wang. *Authorization Revocation for Long-Running AI Agents: Root-Scoped Quiescence under Delegation and Asynchronous Execution*. arXiv:2609.21284v1, 2026.

[6] Lifei Liu, Haoran Yu and Xiaochong Jiang. *VERA: Authority-Preserving Edge Revocation for Federated AI-Agent Workflows*. arXiv:2608.30091v1, 2026.

[7] Jason Liu. *When May an Agent Stop? Evidence-Carrying Termination for Tool-Using LLMs*. arXiv:2608.23623v1, 2026.

[8] LangChain. *Open-SWE*, pinned source commit e0d9aff59925a4da55ec8d6c31fa723651c24909.

[9] Huseyin Buldurgan. *Verifying Stop Scope in Agent Workflows*: Apart sprint project and original evidence v1.0.0, 2026.

[10] Huseyin Buldurgan. *Stop Scope: research progress and evidence snapshot, 16 October 2026*. Versioned release and artifacts, 2026.

[11] Alex Turner. *Why I left Google DeepMind*. Essay.